

Teaching Students Scrum using LEGO Blocks

Maria Paasivaara
Department of Computer
Science and Engineering
Aalto University, Finland
maria.paasivaara@aalto.fi

Ville Heikkilä
Department of Computer
Science and Engineering
Aalto University, Finland
ville.t.heikkila@aalto.fi

Casper Lassenius
Department of Computer
Science and Engineering
Aalto University, Finland
casper.lassenius@aalto.fi

Towo Toivola
F-Secure Oyj
Helsinki, Finland
towo.toivola@f-secure.com

ABSTRACT

In this paper, we present a LEGO-based Scrum simulation game that we used twice with Master's level students at Aalto University. The game was initially developed as an internal training tool in F-Secure Corporation, a Finnish security software company, to support their agile adoption. In the game, student teams learn the Scrum roles, events and concepts in practice by simulating several development Sprints, while incrementally planning and building a product of LEGO blocks. Student satisfaction was measured by a survey at the end of the course, and student learning evaluated by learning diaries. Our results show that the students were highly satisfied with the game, and that students with various degrees of experience with Scrum all learned a lot. In particular, students reported gaining insights about requirements management and customer collaboration, effective teamwork, and the Scrum roles.

Categories and Subject Descriptors

K.3.2 [Computers And Education]: Computer and Information Science Education; D.2.9 [Software Engineering]: Management—*programming teams, software process models*; K.6.3 [Software Management]: Software process

General Terms

Human Factors, Management

Keywords

Scrum, simulation game, software engineering education

1. INTRODUCTION

During recent years, agile methods, such as XP [12] and Scrum [12] have become increasingly popular in the software

industry, as companies want to reap the claimed benefits of these methods, e.g., improved performance and fast response time. As university level teaching aims to provide trained professionals to the industry, the software engineering curricula must respond by providing students with knowledge and experiences on how to use agile methods in practice.

Simulation games have proven to be a good alternative to traditional lecture based teaching, as when well-designed, they are both motivating and effective for learning practical knowledge. There do exist already a good number of educational games for teaching different areas of software engineering. However, the learning effects of those games are often not systematically studied.

Scrum, is one one of the most popular agile methods in the industry [31]. We have taught Scrum at Aalto University for several years, both using traditional lectures combined with expert visitors from industry, and by employing Scrum principles in our capstone project course. However, in 2012 we introduced a Scrum simulation game instead of lectures, a change which received excellent feedback from the students. Even though the basic principles and ceremonies of Scrum are easy, their application is not, as the developers of Scrum state on the first page of the Scrum Guide [25]. Thus, the idea of the simulation game was to practice the application of Scrum before using it on a real project.

In this paper, we present a Scrum simulation game that was first developed as a company internal training tool in F-Secure Corporation to support their agile adoption, and our experiences in using that game at Aalto University.

The paper is structured as follows: Section 2 discusses previous literature on games in software engineering education, Section 3 describes our research methods, Section 4 presents the game, Section 5 discusses the learning results, Section 6 presents improvements to the game, and Section 7 concludes the paper.

2. GAMES IN SE EDUCATION

2.1 Overview of Educational Topics

Games have been employed to assist in teaching of a variety of software engineering topics. The most common topic has been general software development project management. Games proposed for teaching this topic include, among others,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

Copyright is held by the author/owner(s). Publication rights licensed to ACM.

ICSE Companion'14, May 31 – June 7, 2014, Hyderabad, India
ACM 978-1-4503-2768-8/14/05
<http://dx.doi.org/10.1145/2591062.2591169>

SESAM [4], Problems and Programmers [1], SimSE [15], and Simsoft [2].

Another popular topic has been games for teaching specific software development life-cycle models. Games include the XP Game [18] for learning Extreme Programming, System Design for learning rapid application development [3], The Kanban Game for learning Kanban [29] and a group of unnamed games for learning the waterfall life-cycle model [23]. In addition to these general topics, there are games for teaching a wide variety of more specialized topics, e.g., the Requirements Collection and Analysis Game [26] for teaching requirements engineering, IT Billionaire [30] for global software development, and U-Test [28] for testing.

2.2 Evaluation Results

The subjective evaluation of the games was nearly universally positive. Most games were found to be enjoyable, engaging and motivating [30, 1, 26, 8, 16, 23, 27]. Problems identified in games included too long duration [1], boring gameplay elements [1], being boring on repeated plays [16], and limitation to specific software life-cycle model [1].

Learning evaluation results from most of the games indicated that the game had assisted in the learning of the topic [23, 26, 2, 27]. However, the controlled experiments indicate that educational games for SE are not more effective in learning than other methods [6, 16, 28, 33]. These results suggest that the main benefit from using games for SE education is the subjective enjoyment and engagement, but the enjoyment and engagement do not directly translate into better learning results compared to more traditional learning methods.

There is some indication that educational SE games may be more effective in reinforcing and concretizing concepts previously learned using other methods than in learning completely new information [1, 6, 2, 16, 27], although the evidence supporting this result is mostly of low quality.

2.3 Suggestions for Educational Game Design

Most of the previous work agrees on the design principles for educational games. The classroom environment limits time which can be spent on playing the game and educational games should be relatively quick to learn and play [1, 15]. In general, educational games should be fun, engaging, challenging and interactive [30, 1, 9]. The complexity of the game needs to be carefully balanced to reach the learning goal without overwhelming the student with details and mechanics [27, 26]. Especially simulation games need to have complex enough simulation to reflect the real system, but should also be manageable, engaging, fun and educationally effective [19, 14, 15]. Educational games should be fair and the relationship between the student's action and the result in the game should be clear [24, 15, 14, 19].

The games-based SE education literature clearly lacks explicit discussion on when competitive or collaborative design should be used. As stated above, research in other fields has found collaborative games to be more effective in learning to solve ill-defined problems [21, 17]. Games-based SE education articles both support and contradict this notion. Most articles that propose a competitive game do not justify the selection of the competitive design [27, 26]. Only two games explicitly justify the competitive design and claim that the competition motivates the players and encourages learning

[30, 1]. Similarly, the collaborative games do not explicitly justify the collaborative design [8, 2].

2.4 Methods for Teaching Scrum

We expect that the most common method for Scrum learning in teaching in tertiary education has been the traditional lecture model (see e.g. [11]), although it is difficult to confirm this without performing an extensive review of SE education. Recently Scrum has gained popularity as the development life-cycle model of software engineering education capstone projects [22, 13, 20]. Rico and Sayani [22] found that the students should learn agile methods before trying to apply them during a capstone course. According to their descriptive case study, having to learn the agile methods during the capstone course caused notable confusion and slowed down the development progress.

We identified four games have been proposed for learning Scrum. Two of these [34, 10] were proposed by agile development consultants and subsequently lack the rigour, reflection or explicit learning goals usually present in the games-based learning research literature. The other two were published in research literature [7, 32].

The Scrum Game [34] is a collaborative board game that illustrates the Scrum concepts and practices. The game is structured much like a traditional board game. The players each have a piece they move along the board in steps indicated by a dice throw. The squares in the different parts of the board represent the sprint planning activities, work during the sprint and impediment management. The goal is to get all work completed before the sprint, and the game, ends.

Scrum Simulation with LEGO Bricks [7] is a collaborative simulation of a Scrum development project. One or more teams consisting of the students collaborate to build a LEGO city. The teacher acts as the product owner. The teacher constructs a backlog of structures for the LEGO city which the students then estimate. The teams conduct sprint planning and then try to construct the structures during the sprint. After the first sprint the teacher reviews the structures and provides feedback, followed by two additional sprints. After the last sprint, the participants conduct an open-ended debriefing session to share learnings and observations from the simulation.

PlayScrum [10] is a competitive card game aimed for university students to learn concepts, practices and practicalities associated with the adoption of Scrum. Each player plays as a Scrum master in a Scrum software development project. The players try to complete their task cards by playing developer cards. The players also try to sabotage the other players by playing problem cards which the other players must try to mitigate using concepts cards. The game claims to teach all Scrum related topics except the burn-down charts and the sprint planning and review meetings. The game was evaluated with a qualitative post-game self-assessment survey answered by 13 master's level students. According to the evaluation results, the students found the game enjoyable and sufficiently challenging. The students thought that the game complemented the lectures and helped to learn about software engineering reasonably well. According to the results the students also thought that the game helped to understand the role of a Scrum master and especially the importance of making appropriate developer hiring decisions.

SCRUMIA [32] is a collaborative simulation of a Scrum project using papers and pencils. The players assume the

Table 1: Data Collection and Analysis

Data collection method	Collected data	Analysis method
End-of-the course surveys	22 answers (2012), 26 answers (2013)	Quantitative analysis
Learning diaries	23 diaries (2012), 18 diaries (2013)	Qualitative coding in Atlas.ti
Interviews	Product Owners (2), Scrum Master (1), Team members (2)	Qualitative coding

role of a team member, Scrum master or product owner. Their task is to create objects from paper sheets during several sprints. The players conduct the whole Scrum process. After the game, the players and the teacher conduct a debriefing session to reflect and share their learnings on Scrum. The game evaluation was based on quantitative post-game self-assessment with 73 students over three semesters and two courses. According to the results, the game reached the learning goals and reinforced and taught Scrum knowledge and application. The students found the game enjoyable and engaging. Although the game was non-competitive, an implicit element of competition between the Scrum teams emerged during the game. This impaired the Scrum learning, as the students concentrated on outproducing the other teams instead of learning Scrum.

3. METHODOLOGY

The goals of this paper are twofold: 1) to present a new Scrum simulation game, and 2) to study the student reaction and learning from the game.

To assess the learning results from teaching Scrum using this game, we used a mixed-method approach combining qualitative and quantitative methods. Each student filled in an end-of-the-course survey and wrote a learning diary on the simulation. To receive industrial comments from the game, we interviewed five persons from F-Secure that had either facilitated or participated in the game. Next, we present our research questions and describe the data collection in more detail.

Our research questions related to the second goal are the following:

- RQ1: What was the student reaction to the Scrum simulation game?
- RQ2: What did the students learn by the Scrum simulation game?

We asked our students to write a learning diary of 1-2 pages on their experience with the simulation game. They were in particular asked to discuss what they learned during the simulation, as well as to compare and critically discuss the "lecture" contents to what they learned during lectures, in other courses, from the literature, from other visitors, as well as to contrast the learnings with their own experiences. On the Software Project Management course, the students wrote a similar learning diary from the "visiting lecture" part of all seven face-to-face meetings, thus the concept was familiar to them. None of the diaries or assignments were obligatory for the students, but they had to gain enough points to pass the course, thus they could leave a few tasks undone, if others

were done well. In 2012, we received 23 diaries, and in 2013 18 diaries. In both years, 23 students participated in the game.

The learning diaries were analyzed using qualitative coding using Atlas.ti. We created the codes by first reading through all the diaries and creating the first set of codes, which we then improved by adding a few additional codes during the actual coding process. Examples of codes include "Product Owner role", "Scrum Master role", "Customer collaboration", "reflection", "game improvement", "reaction to game".

We required all students to fill in a course feedback survey at the end of the course. In this survey we asked the students to rate all the lectures by course personnel, by visiting lecturers, all the assignments, as well as special exercises during the lectures, e.g. a planning poker exercise.

As we wanted to understand both why and how the game had been developed, as well as gather F-Secure's own experiences of the game, we interviewed five persons from F-Secure: the person who developed the game and participated in both our games as a Product Owner and game leader; the other Product Owner of the first game, who worked as an improvement coach; a Scrum Master who participated in both our games, as well as some previous games at F-Secure and who had worked several years as a Scrum Master; a process improvement team member, who participated in our second game as a team member, as he wanted to update his Scrum knowledge; as well as another process improvement team member who had participated in one of the company internal games as a team member. The game developer's interview took one hour, and the others 30 minutes each. The interviews were semi-structured to give enough flexibility to discuss emerging topics, and were recorded and transcribed by a professional transcription company. We analyzed the interviews using qualitative coding.

4. THE SCRUM SIMULATION GAME

This section presents the Scrum simulation game, fulfilling the first goal of this paper.

4.1 Game Background

The Scrum simulation game, called the Scrum LEGO Challenge, has been developed by F-Secure Corporation for their internal use, to teach their teams Scrum. F-Secure is a software product company. Their offerings include PC computer security software, as well as mobile security and data security software. Currently, the company employs around 1000 persons and has subsidiaries around the world. Their adoption of the Scrum method begun in 2006 with the goal to improve the efficiency and decrease the lead-time of the software development.

Originally, the company had a Scrum training that had both lectures and a practical game type of part, where a paper swing was build. The fourth author, who at that time was F-Secure's Software Process Improvement Manager improved the training further by developing a LEGO game with clear similarities to software engineering. The first instance of the game was played by the F-Secure's internal process improvement group to test it, especially the timing of the different parts. The game was initially taken into the use in development teams during 2010. As the Scrum adoption had started already a few years earlier, this game was used more like a refresher and a team building tool. The game designer aimed for a game that would be suitable to

persons with different levels of Scrum experience and would provide value to all participants. For example, teams already applying Scrum but having problems, or applying Scrum only partially would get new insights on what they could do differently.

I aimed to ... have all essential parts of Scrum, as I cannot know where the audience has challenges, as this was meant for an internal training, ... so we take all the parts as a balanced set, so wherever they have problems, they will get some new insights on that. — The Main Product Owner of the game

The company internal games were played by one team at the time, but during a Software Engineering Day arranged with collaborating companies the game was played successfully with three teams. We first heard about the Scrum game while conducting interviews in a joint research project studying F-Secure's agile adoption, and subsequently invited them to play the game with our students in the Spring of 2012. As the game received excellent feedback from the students, we reinvited them the next year.

4.2 Course Background

The Scrum LEGO Challenge was played as part of the Software Project Management course at Aalto University during two instances of the course, during spring 2012 and spring 2013. The course is aimed at Master and post-graduate level students as a basic course on software project management. Most of the students have software engineering as a major or minor and around half of them have work experience from industrial software engineering projects.

The course consists of seven weekly face-to-face meetings of 3-4 hours, as well as weekly assignments and learning diaries based on visiting lectures. Each face-to-face meeting has a specific topic, such as software process models, effort estimation, distributed software development, managing people, and lean and agile transformation. During 2012 and 2013 the Scrum LEGO Challenge has been the topic of one of these face-to-face meetings.

As the game was played during the fourth (2012) or third (2013) meeting, the students were already familiar with software process models and Scrum, as they had read the Scrum Guide [25].

4.3 Learning Goals

The main learning goals of the Scrum simulation game were: 1) Scrum process and roles, 2) requirements management and customer collaboration, 3) estimation, 4) working in a team, and 5) Visualizing work and progress

4.4 Game Description

4.4.1 Set-up

On both instances of the course, the students were divided into three Scrum teams of 7-8 persons, each team organized around a team table. F-Secure provided each team a professional Scrum Master, who prepared the Scrum boards for the teams. The Scrum boards included a product backlog, a sprint backlog, a sheet for team velocity calculation, as well as a burn-up chart to follow progress (see Figure 3). Boards, drawn on paper sheets, were set up to walls close to the team tables.

During the first instance of the game we had two Product Owners. The main Product Owner (the person who had developed the game) was leading the game and working as a

Product Owner for two teams, and the other Product Owner worked with one team. By having two Product Owners we wanted to ensure that the teams do not have to wait too long for the Product Owner. However, we had noticed during the first game that the teams were progressing at a different pace, so they did not always need the Product Owner at the same time, and other hand, in real life Product Owner's time is a rare resource. Thus, in the second game we had only the main Product Owner working with all three teams, which succeeded well.

Moreover, each team was provided with different color post-its and pens, as well as a box of LEGOs, including different kinds of blocks that well-used would be more than enough to complete the task.

4.4.2 Project

All the teams were given the same task, a project called "Mushroom", which aimed to build a new product to the markets, an Antarctic exploration set including luxury cabin, sauna, helicopter, helipad, snow scooter exploration vehicle, fence and snow plow. The company aimed for the first General Availability Release (GA) to be ready after three iterations. The fourth iteration was optional and would be used if there was still time.

4.4.3 Schedule

The game was divided into three phases: 1) introduction to the project and the way-of-working, including the project goals, 2) the simulation game, including release planning and 3-4 iterations, and 3) reflection discussion on learning. The game flow and schedule is described in Figure 1.

During the first phase, the game leader presented the game structure, goals, the way-of-working as well as guidelines and instructions. The instructions included explanation on some limitations due to the game structure compared to real-life Scrum, such as "Overhead to work ratio is absurd, due to simulation", "Do not touch the code outside Sprint time", "This is not the optimal way to build LEGOs" and "Due to pedagogics, do not talk while coding and testing".

The second phase was the simulation game part. The simulation started by the release planning during which the teams created the product backlog by sticking post-its to the backlog board on the wall, with help of the Product Owner. Next, each team planned the first iteration: in sprint planning part I, they planned with the help of the Product Owner "WHAT" and "WHY" would be build, and then in sprint planning part II, the team planned "HOW" they will do it. Iteration planning included splitting backlog items into tasks, estimating the tasks and deciding the committed number of story points for that iteration. Each iteration had two work passes and a Daily Scrum meeting between them, which would serve as a checking and coordination point during the iteration. In the end of the iteration each team would demo their results to the Product Owner, who would test the results, give feedback and accept the features that he deems "done". Finally, the team would have a retrospective meetings, during which they would discuss their way-of-working in the previous iteration and decide how to improve, e.g. what to "keep", "drop" or "try" in the next iteration. As all parts of the iteration were not timed, e.g., iteration planning part I and demo, during which the Product Owner visited the teams, the teams could proceed on a somewhat different pace and have short breaks while waiting for the Product

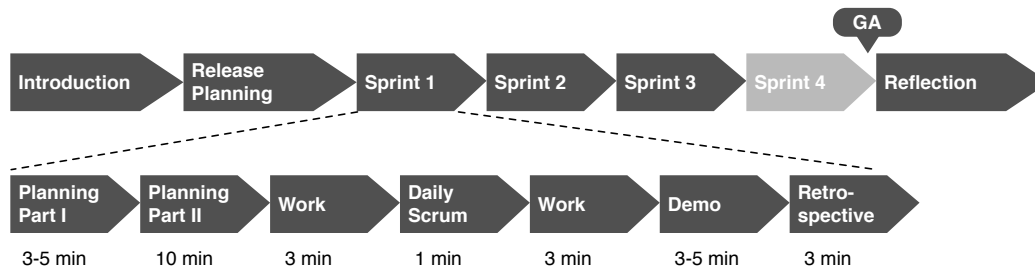


Figure 1: Game flow

Owner to arrive from the previous team. Thus, depending on the team, each team was able to work through three to four iterations of this kind before the game was stopped and it was time for each team to present their final, releasable product to the Product Owner and the other teams.

The third and final phase was reflection discussion on the game, that was lead by the main Product Owner. During the reflection students were encouraged to explain what they felt they learned and ask questions and clarifications, whereas the Product Owner explained from his point of view what he hoped the students would remember from this simulation.

4.5 How Does the Game Aim to Accomplish the Learning Goals?

Next we explain how the game design aims to accomplish the learning goals.

4.5.1 Scrum Process and Roles

First of all, the game reveals all the steps of the Scrum process and related Scrum ceremonies: what they are, and how to do them in practice. As the game has 3–4 iterations, these process steps should become quite familiar to the participants. The game aims to show the iterative nature of the development, how more features can be added iteration by iteration, while getting feedback from the Product Owner regularly.

The main Scrum roles, the Product Owner, Scrum Master and the team, and their responsibilities come clear during the game. Honoring the time-boxes is emphasized by struck timing of the phases.

Scrum artifacts, like product and sprint backlogs are created used together in each team.

4.5.2 Requirements Management and Customer Collaboration

Requirements management and the importance of customer collaboration is emphasized by the game. The leader of the game, playing the main Product Owner is rather stereotypical, by acting as a real business person, talking about business goals, not understanding technical issues, having hard time in expressing what he really wants and in prioritizing and even changing his mind.

As a Product Owner I do not understand about LEGOs.... They have to discuss the business goals with me. I try to create the difference between those two worlds of using language, but not so that it is obvious.... You have to create the feeling that we understand each other after the sprint planning, like it normally is... and then take care that they have not really understood. And then, at some point they understand to start asking relevant questions, and then that is a victory, they have learned

something when the team starts asking questions.

— The Main Product Owner of the game

By creating suitable misunderstandings the Product Owner tries to challenge the team members to really collaborate with the customer by asking questions, making suggestions and not agreeing to deliver everything he want, but only what is possible when taking into account the time constraints.

By working with the Product Owner, the teams learn the importance of prioritizing. They practice demoing and receiving and reacting to feedback. During demos the Product Owner test the product, which shows the team the importance of early integration and proper testing. For example, the Product Owner may test whether the snow scooter will really fit the shelter:

Then the vehicle shelter.... so typically where it goes wrong, is when I try to drive the vehicle there. Since one guy has build the vehicle and the other one the shelter. So if it mystically would happen that I would be able to drive it there, if I then add the drive to the vehicle and drive it to the shelter, the drives forehead hits the roof. "Guys, not like this, the nearest hospital is in 14 000 km!" ... I try to underline that integration is important. Integrate and test from the end users point of view!

— The Main Product Owner of the game

Actually we could say that the Product Owner role is the most important role in the whole game, thus he needs to be quite experienced with software development projects, plan his role well, and have some acting skills.

Product Owner role is a key role in this game it is important that the Product Owner is a person, who has seen a good number of software development projects, so that he knows when these misunderstandings do happen. Because with LEGOs the misunderstandings do not happen so easily, that you have to insert on purpose those misunderstandings in there.

— The Main Product Owner of the game

4.5.3 Estimation

In the beginning of each iteration the team practice estimation, while splitting and estimating together the tasks for the next iteration. While comparing how well the previous estimates were realized and whether they could really achieve the number of story points they committed to, they can improve their next estimates and do more realistic commitments. With help of their Scrum Master the teams learn to calculate their velocity and use it while planning the next iteration.

By prioritizing, estimating, and calculating velocity the teams can learn how to work with the time-boxes.



Figure 2: Teamwork and End Result

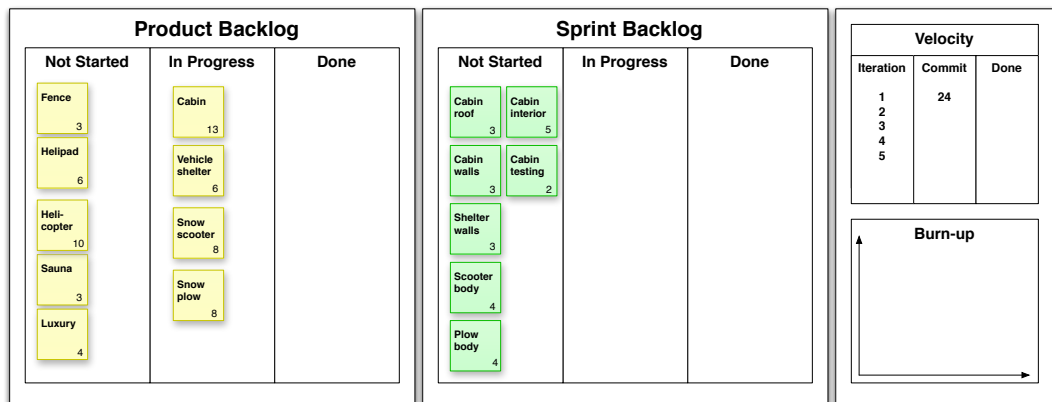


Figure 3: Scrum board

4.5.4 Working in a Team

As the teams are size of real Scrum teams, seven to eight persons, the students practice collaborating as a team. The Scrum Master coaches the team and challenges the team to ask questions and to think.

I had to challenge them to think, to find right questions... that the Product Owner is coming and stays a limited time, how do we get the most essential information? ... I challenged them to think, whether we have really understood what we are doing? ... I opened up discussions... that people would think themselves what the challenges are. The LEGOs are actually a good analogy, because everybody thinks it is easy... but the time limit ... and taking into account the customer creates the biggest challenge.

— Scrum Master

The Scrum Master role is another essential role in the game, as he really needs to understand the Scrum Master role to be able to behave like a real Scrum Master and not as a team leader. Thus, the Scrum Master should remember his role as a coach, by not telling the team what to do, but by challenging the team by asking right question.

All team members encouraged to actively participate in planning and communicating in a team. They really need to collaborate for their project to succeed. While using LEGOs it takes all student to a pretty same level regarding their coding skills, as everybody can build LEGOs! Thus, participation in the team work is not limited due to different skill levels, but all can be active contributors.

Finally, each team should spend some time during the retrospective on thinking back to their previous iteration, reflect their way-of-working and decide how to improve. This is another challenging moment for the Scrum Master to make the team step on a higher abstraction level for a while:

It (retrospectives) is realistically challenging in this game, it doesn't differ from the day to day work, that most people would rather do something else than think how they did the previous work... the challenge is to step for a while to another abstraction level.... but without exception we came to that, we could direct the discussion on how to do the work on a general level, and not only discuss LEGOs.

— Scrum Master

4.5.5 Visualizing Work and Progress

Finally, our last learning goal of visualizing work and progress is accomplished in the game by creating the Scrum boards for each team. The boards include the product and sprint backlog, burn-up chart and velocity calculation. The work and progress is visualized by backlog items written and estimated to post-it notes, that are moved as the work proceeds. The burn-up chart and velocity is calculated by the team with help of their Scrum Master after each iteration.

5. EVALUATION

In this section we discuss how the students reacted to the game and what students reported learning during the game, fulfilling our second goal for this paper. As a framework for evaluation learning we used Kirkpatrick's 4-level framework for evaluating training programs [5]. Kirkpatrick's levels are: 1) reaction, 2) learning, 3) behavior and 4) results. Reaction, evaluates how the students felt about the training or learning experience. A positive reaction to the game might not ensure high learning results, but an attractive and motivating game at least keeps the students engaged, while a negative reaction easily makes students unmotivated and prevents learning. Learning is measured by the increase in knowledge. Behavior is evaluated on the extent of applied learning when back on the job. Results is the evaluation on the effect of the

business or environment by the trainee. We used levels one and two for evaluating the game, as the higher levels are only applicable when students start working in professional settings.

5.1 Reaction

We evaluated the student reaction (Kirkpatrick's Level 1) by the end-of-the course survey and by learning diaries. The survey results presented in Figures 4a and 4b clearly show that the students rated the Scrum simulation very high, most students giving it the best possible grade. It received an average of 4.82/5 (2012) and 4.80/5 (2013) on a Likert scale of 1–5 (1=worst, 5=best). When compared to other learning methods employed during the course we can see that the simulation game is the highest ranked element of the whole course. The other elements included, e.g., a practical planning poker exercise during a lecture and a company visit during which we could hear and see a huge lean and agile transformation in a telecom company. In addition we included the general grade on all the lectures, including visiting lectures.

Based on the learning diaries, the overall reaction was very positive. Students explained that the game was both fun and that they learned a lot. Many explained that they liked this way of learning, where they could immediately apply their learning in practice instead of just sitting in lectures. Several students even mentioned that they would recommend this kind of training to their companies, or would like to arrange a similar training when working in a project in the future.

We were surprised that the reaction to the game was so positive both among the non-experienced students and those having working experience. Both felt they gained new knowledge from the game, even though their learning concentrated on somewhat different areas: non-experienced students learned the basic Scrum terminology and principles, while the experienced students reported that they had understood new things e.g., regarding customer collaboration or teamwork. A few of the students had even worked as Scrum Masters in real projects previously. One student, having passed a certified Scrum Master training arranged by the Scrum Alliance, even commented that he found this train-

ing useful and was wondering why the 2-day Scrum Master training did not include this kind of practical part.

5.2 Learning

Next, we describe what the students reported they learned (Kirkpatrick's Level 2), regarding each of the learning goals, during the simulation based on the learning diaries.

5.2.1 Scrum Process and Roles

As the Scrum phases in general were known to the students before the simulation, the students did not comment on them so much, even though especially understanding what happens in reality in sprint planning and in retrospective were seen as new learning experiences. However, really understanding the Scrum roles and their tasks, Product Owner, Scrum Master and the team seemed to be a major lesson learned for many. It seems clearly that the game had opened these roles to many, as the following quotes illustrate:

... I was surprised that the Scrum Master role was like a coach. I had understood previously that the Scrum Master would be more like a project manager instead of a coach or a mentor.

— Student, 2012

... the experiment made me realize that in scrum, there isn't any traditional project manager that would plan the work and demand constant changes for the product but instead, the responsibility of development has really been given to the development team.

— Student, 2012

This game was also an interesting experience in understanding the general attitude of the product owner. The Product owner is interested in the delivered prototypes or products at the end of each sprint and does not focus on the actual design and development phases.

— Student, 2012

5.2.2 Requirements Management and Customer Collaboration

Requirements management and collaboration with the customer was clearly the topic that the students discussed in their learning diaries the most. It was both a new area to the students, and as the Product Owner was in a really central role in the game, collaborating with the teams, he could give the participants many opportunities for Aha-experiences, which they described in their diaries and would remember

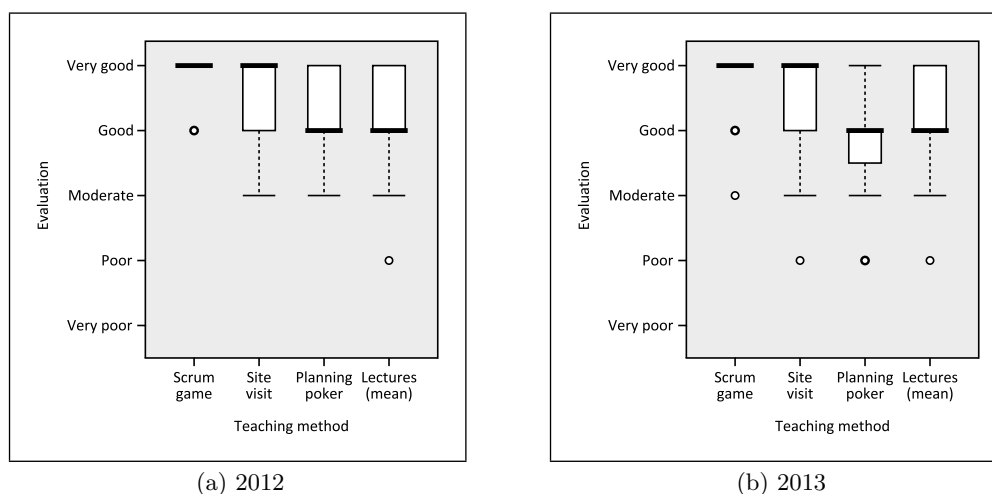


Figure 4: Student evaluation of the different learning methods.

for a long time during their carriers. Students learned how dangerous it is to make assumptions and not really discuss with the customer what he really wants.

For me, the most interesting thing in the exercise was to see how difficult it can be to understand requirements. I constantly noticed how I was making assumptions on how to implement things or what the priorities are. Many times the product owner's opinions were complete opposites of what I initially had thought of. Further, demonstrating the results of iterations brought up new ideas for the product owner, so the requirements were changing. These findings remind me that one should never make assumptions, but focus on customer collaboration.

— Student, 2012

I think our team posed a large variety of questions regarding the size, look and details of the items but we didn't start to question our core assumptions. However, it happened that the product owner viewed some of the basic things differently. . . . If this is translated to software development, understanding some basic element of the product wrong and not realizing it before than at the end of the project will require refactoring the whole product. The experiment made me question even more the traditional software development methods in which customer is little involved in the software development phase.

— Student, 2012

After the first confusion, the team learned how to negotiate with the customer and not to take for granted everything he says regarding the requirements, scheduling, or the way of fulfilling the requirements. They learned how to split requirements into smaller pieces and to first implement only the "must have" part, e.g., some teams realized to ask the Product Owner, whether the delivery of the "Luxury cabin" could be divided into two parts, first delivering just a normal cabin and possible in the later iterations adding luxury. In addition, some teams innovated new ways that some customer's needs were fulfilled and customer was highly satisfied, even though the team did not do exactly what the customer had asked for in the first place. For example, one team build the helipad over the sauna, which would melt the snow from the helipad and a snow plow would not be needed. Another team integrated all the buildings so that the fence that the customer asked to protect the humans from polar bears, was not need:

. . . our team did not know or understand to ask the right question. . . later on, as we progressed in the development, we had the courage to question the initial guidelines of the customer. For instance, we, . . . decided to combine all components together as one big complex structure and leave the fence out entirely, there was no need for it after all. . . we learned that it is central to listen closely what the customer says while not necessarily taking everything for granted,

— Student, 2013

We could find many this kind of stories from the learning diaries, which showed that these discoveries had really been major learning experiences for all team members.

5.2.3 Estimation

Regarding estimation, the students learned how to set a scale for estimation and how they can estimate as a group. The teams could themselves decide how to do the estimation in practice. During the second instance of the game part of the teams decided to use planning poker, as we had had a planning poker exercise during the previous lecture. This showed that the students found planning poker as valuable and deepened their knowledge on applying it during the game. However, other methods were used as well:

Our team learned as in general that the later in the project you create estimates the more accurate these estimates are. This

was what happened to us. I remember the first sprint's estimations were pretty low. . . Our estimation techniques used was the analogy. . . It seemed natural after some tasks been done to compare to those tasks when a new story had to be estimated.

— Student, 2013

Besides estimating, the teams practiced how much they can commit to, how to monitor their progress, and if needed they learned to improve the estimates in future iterations.

According to the burn-up our team was able to perform quite well. One reason might have been not over-committing, thus we could deliver what we promised in every sprint. In planning poker we used man minutes and committed 75 % of available resources.

— Student, 2013

5.2.4 Working in a Team

Several students mentioned teamwork and communication within the own team as the most important things they learned. Some of them analyzed how they had improved their team work and working practices iteration by iteration and a few students even compared their team maturation to the Tuckman's stage of team formation.

The first work period was a total chaos. . . During the second iteration planning we improved our planning tactics. . . For the third and final sprint we decided to work more independently and integrate everything during the second work period of the sprint. Even though integration is somehow a risky approach, the tactic paid off and we were able to integrate all components successful and deliver. . . Personally I was happy about how this team developed.

— Student, 2013

The teamwork during the game was pretty intensive. During the sessions, we noticed that teams with students inexperienced in working in teams had more difficulting with collaboration than teams with more experienced students. The lecture diaries confirmed this observation:

During this lecture I learnt a lot about teamwork. I had to accept that I'm not always right. My team was international with different people, with different attitude. I had to realize that we all look at the problem in a different way. Everybody had to accept the compromises. Sometimes it is good to just listen and learn from others, adapt to the situation and let others persuade you.

— Student, 2013

5.2.5 Visualizing Work and Progress

The teams learned in practice how to use the Scrum boards: fill in the backlogs and follow the progress from the burn-up.

. . . I had not perfectly understood the purpose of product and sprint backlog, but now when we could use them in practice I learned their purpose. One new thing that I learned about Scrum was just that the team can decide in sprint planning how big part of the product backlog will be taken to the backlog of the next sprint. . .

— Student, 2013

However, this kind of comments were found only from a part of the learning diaries, which shows that part of the students either did not learn much about this learning goal or did not find it as important as the other topics they learned.

6. IMPROVING THE GAME

In this section we discuss how the game has been further developed between the two instances of the course and what we see as the main future improvement areas.

The students of the first instance of the game were asked to add comments and improvement suggestions for the course to their lecture diaries. We gathered these comments and provided them to the game leader, who took them into account

while planning the second instance. The main improvement suggestion was to increase the time available for the game so as to have more time for reflection discussion after the game, since many students saw the discussion as the most valuable part of the learning experience. During the first instance we had 2h 45 min for the game. For the second instance, we added one hour, giving a total of 3h 45 min, which proved to be optimal, leaving enough time both for playing the game and for post-game reflection.

The students also wanted us to improve the instructions in the beginning of the game, in particular to make the game flow more clear to the participants before starting the first iteration. In addition to this, the game leader added some slides with "immediate wisdom" between each iteration, so that students could reflect on them during the next iteration. These included advice on "doing active backlog work", "keeping promises and promising what you can keep" and "keeping the customer in mind". These improvements seemed to be well-received.

Minor improvement suggestions included the question why the team members were not allowed to discuss while coding. In the next instance of the game, the game leader explained the reasons of that rule, e.g., that this rule brings it visible how important constant communication is.

There are a few topics that we aim to improve in the future. The use of the boards for visualization and monitoring progress probably remained unclear to some, as in most of the teams it was the Scrum Master who updated the boards. In the future, we plan to explain the rationale for, and usage of these boards in the game introduction.

The Daily Scrum meetings were only one minute long, which turned out to be a too short timebox for 8 persons to answer the three Scrum questions. Thus, we plan to add a few more minutes for this activity in the future.

7. DISCUSSION AND CONCLUSIONS

In this, paper we presented a new simulation game for teaching Scrum. We evaluated to method by collecting data from two games conducted with Master's level students in the form of surveys and lecture diaries. Based upon our results, we conclude that the Scrum simulation game is a valuable tool for teaching university students the Scrum method in practice. The student reaction to the game was highly positive, and based on the learning diaries the students learned a lot. When comparing our learning goals to what students really learned, we see that the students learned more than we expected regarding requirements management and customer collaboration, effective teamwork and the Scrum roles. In the lecture diaries we could see how in particular Scrum roles had been unclear to many before the game, while just reading about them from the literature.

Regarding requirements management and customer collaboration, our Product Owner was in a key role, as he could really simulate a stereotypical business person and insert planned confusions into the game, bringing several great learning opportunities for the students in the form of Aha-experiences. Another critical role was the Scrum Master, as he/she should really know his/her role by heart and coach the team to ask questions and discuss relevant things to learn all the time. As all our Scrum Masters were professionals, their crucial role performance was perfect.

Regarding effort estimation, we noticed better learning results in our second game instance, as then the effort estima-

tion lecture with the planning poker exercise had taken place before the simulation, which clearly helped the students to deepen their learning on the application of their knowledge during the game.

Our last learning goal, visualizing progress, was the one that clearly received less attention in the learning diaries. Even though as a learning goal it was clearly less important than the other, we learned that this is one part that we could improve in the future, e.g., better explaining before the game the role and usage of the boards.

Even though the students participating in the game were a heterogeneous group regarding background knowledge on software development and Scrum, some had a several years of working experience and a few had used Scrum professionally, while some did not have any work experience, they all felt that they learned new during the game. The most inexperienced learned about the basic terminology and the Scrum phases, while the most experienced had those Aha-experiences on how to apply Scrum in practice. However, as the game was originally designed for professional software developers, it is essential that the game participants have at least some background knowledge on software engineering and basic knowledge on Scrum.

Overall, the game format is simple and LEGOs are cheap tools. LEGOs actually brought smiles on the face of the students when they entered the room, and thus they had a positive attitude right from the beginning. LEGOs also put everybody on the same level regarding their coding experience, as everybody can build LEGOs. Thus, students could focus on learning Scrum and everybody in the team had equal possibilities to contribute.

The game managed to simulate the phases and difficulties of real software projects surprisingly well. Both students and the interviewed industrial participants commented on this.

A limitation of our evaluation of the game was that we did not have a before-the-course data collection element to assess the knowledge level of the students before the game to be able to compare the real learning during the game. Other limitations are that we did not have a control group and that the learning diary is a subjective self-assessment instrument, in which students might exaggerate their learning to please the teacher. However, as the learning diaries did not list topics that the students had learned, like a survey assessment would do, the learning diary had two benefits compared to the surveys: first, students mention only topics they felt they learned, and second, as many students explained in detail what, why, and how they learned by giving examples from the situations of the game, the reader could really see from the text what the student had understood. Therefore, in spite of the limitations, we believe that our results show how the game advanced student learning.

In the future we aim to use this game on the same course again. In addition, we are planning to use it as part of a Software Engineering training aimed for professionals who want to update their knowledge on agile software engineering.

8. ACKNOWLEDGMENTS

We would like to thank F-Secure personnel, especially Tovo Toivola, for developing this Scrum Lego game, spending time to share the game with us, play it with our students and providing us an opportunity to collaborate while writing this paper. Finally, we would like to thank all interviewed persons and the students who participated in the games.

9. REFERENCES

- [1] A. Baker, E. O. Navarro, and A. van der Hoek. An experimental card game for teaching software engineering processes. *Journal of Systems and Software*, 75(1–2):3–16, 2/15 2005.
- [2] C. Caulfield, S. P. Maj, J. Xia, and D. Veal. Shall we play a game? *Modern Applied Science*, 6(1):2–16, 2012.
- [3] W.-C. Chang, Y.-L. Chen, and T.-P. Lee. Computer assisted learning with card game in system design concept. In *Advances in Blended Learning*, volume 5328 of *Lecture Notes in Computer Science*, pages 93–101, 2008.
- [4] M. Deininger and K. Schneider. Teaching software project management by simulation – experiences with a comprehensive model. In *Software Engineering Education*, volume 750 of *Lecture Notes in Computer Science*, pages 227–242, 1994.
- [5] K. D.L. and K. J.D. *Evaluating Training Programs: The Four Levels*. Berrett-Koehler Publishers, 2006.
- [6] A. Drappa and J. Ludewig. Simulation in software engineering training. In *Proceedings of the 22nd international conference on Software engineering (ICSE '00)*, pages 199–208, New York, NY, 2000. ACM.
- [7] J. M. Fernandes and S. M. Sousa. Playscrum - a card game to learn the scrum agile method. In *Proceedings of the Second International Conference on Games and Virtual Worlds for Serious Applications (VS-GAMES 2010)*, pages 52–59, 2010.
- [8] T. Hainey, T. M. Connolly, M. Stansfield, and E. A. Boyle. Evaluation of a game to teach requirements collection and analysis in software engineering at tertiary education level. *Computers & Education*, 56(1):21–35, 2011.
- [9] E. Knauss, K. Schneider, and K. Stapel. A game for taking requirements engineering more seriously. In *Proceedings of the Third International Workshop on Multimedia and Enjoyable Requirements Engineering – Beyond Mere Descriptions and with More Fun and Games (MERE '08)*, pages 22–26, 2008.
- [10] A. Krivitsky. A multi-team, full-cycle, product-oriented scrum simulation with lego bricks – the small & medium business edition v2.0. <http://www.lego4scrum.com/p/scrum-with-lego.html>, October 2011.
- [11] P. Kruchten. Experience teaching software project management in both industrial and academic settings. In *Proceedings of the 24th IEEE-CS Conference on Software Engineering Education and Training (CSEET 2011)*, pages 199–208, 2011.
- [12] C. Larman. *Agile and iterative development : a manager's guide*. Addison-Wesley, Boston, MA, 2004.
- [13] V. Mahnič, S. Georgiev, and T. Jarc. Teaching scrum in cooperation with a software development company. *Organizacija*, 43(1):40–48, 2010.
- [14] P. Mandl-Striegnitz. How to successfully use software project simulation for educating software project managers. In *Proceedings of the 31st Annual Frontiers in Education Conference*, volume 1, pages T2D–19–24, 2001.
- [15] E. O. Navarro and A. van der Hoek. Software process modeling for an educational software engineering simulation game. *Software Process: Improvement and Practice*, 10(3):311–325, 2005.
- [16] E. O. Navarro and A. van der Hoek. Comprehensive evaluation of an educational software engineering simulation environment. In *Proceedings of the 20th Conference on Software Engineering Education & Training (CSEET '07)*, pages 195–202, 2007.
- [17] D. Nembhard, K. Yip, and A. Shtub. Comparing competitive and cooperative strategies for learning project management. *Journal of Engineering Education*, 98(2):181–192, 2009.
- [18] V. Peeters and P. V. Cauwenberghe. Xp game - the manual. <http://www.xp.be/xpgame.html>, 2008.
- [19] D. C. C. Peixoto, R. M. Possa, R. F. Resende, and C. I. P. S. Padua. Faseng: A framework for development of software engineering simulation games. In *Proceedings of the 2012 Frontiers in Education Conference (FIE 2012)*, pages 1–6, 2012.
- [20] M. Persson, I. Kruzela, K. Alder, O. Johansson, and P. Johansson. On the use of scrum in project driven higher education. In *Proceedings of the World Congress in Computer Science, Computer Engineering and Applied Computing*, 2011.
- [21] Z. Qin, D. W. Johnson, and R. T. Johnson. Cooperative versus competitive efforts and problem solving. *Review of Educational Research*, 65(2):129, 1995.
- [22] D. F. Rico and H. H. Sayani. Use of agile methods in software engineering education. In *Proceedings of the 2009 Agile Conference (AGILE '09)*, pages 174–179, 2009.
- [23] A. Rusu, R. Russell, J. Robinson, and A. Rusu. Learning software engineering basic concepts using a five-phase game. In *Proceedings of the IEEE Frontiers in Education Conference (FIE 2010)*, pages S2D–1–S2D–6, 2010.
- [24] B. A. Scharlau. Games for teaching software development. In *Proceedings of the 18th ACM conference on Innovation and technology in computer science education (ITiCSE '13)*, pages 303–308, New York, NY, 2013. ACM.
- [25] K. Schwaber and J. Sutherland. The scrum guide – the definitive guide to scrum: The rules of the game. <http://www.scrum.org/Scrum-Guides>, October 2011.
- [26] R. Smith and O. Gotel. Gameplay to introduce and reinforce requirements engineering practices. In *Proceedings of the 16th IEEE International Requirements Engineering Conference (RE '08)*, pages 95–104, 2008.
- [27] G. Taran. Using games in software engineering education to teach risk management. In *Proceedings of the 20th Conference on Software Engineering Education & Training (CSEET '07)*, pages 211–220, 2007.
- [28] M. Thiry, A. Zoucas, and A. C. da Silva. Empirical study upon software testing learning with support from educational game. In *Proceedings of the 23rd International Conference on Software Engineering & Knowledge Engineering (SEKE'2011)*, 2011.
- [29] Y. Tsutomu. The kanban game. <http://groups.yahoo.com/neo/groups/kanbandev/conversations/topics/9216>.
- [30] R. van Solingen, K. Dullemond, and B. van Gasteren. Evaluating the effectiveness of board game usage to teach gse dynamics. In *Global Software Engineering (ICGSE), 2011 6th IEEE International Conference on*, pages 166–175, 2011.
- [31] VersionOne, Inc. 7th annual "state of agile development" survey. <http://www.versionone.com/pdf/7th-Annual-State-of-Agile-Development-Survey.pdf>, 2013.
- [32] C. G. von Wangenheim, R. Savi, and A. F. Borgatto. Scrumia—an educational game for teaching scrum in computing courses. *Journal of Systems and Software*, 86(10):2675–2687, 10 2013.
- [33] C. G. von Wangenheim, M. Thiry, and D. Kochanski. Empirical evaluation of an educational game on software measurement. *Empirical Software Engineering*, 14(4):418–452, 2009.
- [34] W. Wake and M. Cohn. The Scrum game. <http://www.mountaingoatsoftware.com/products/scrum-game>, April 28 2007.